

The Retry Read

Assignment #6, GER pipeline. Rex Machina capstone.

A Generator, Evaluator, Refiner and Circuit Breaker that write the line the boss speaks when the player respawns after losing.

Run it with `python pipeline.py`. Nothing to install.

Pre-Build Declaration

What content does my game generate manually, inconsistently, or not at all? The retry read. When the player runs out of stamina she respawns at the last phase gate, and GDD §3 promises that the first thing the robot says references the run she just lost. The game has none of these lines. My Assignment 5 agent scanned the codebase and listed `prior_attempt_read` as absent.

What rule from my GDD must every one of them satisfy? GDD §3, exploit 1, the read-line oracle: past-tense diagnosis only, never intent. The robot reports what it measured. It never announces what it will do.

What does a failure look like in my game's terms? The player respawns and the robot says "I'll seal the fence gap this time". She now knows the plan before committing a move, counters it for free, and the fight becomes unloseable. That is the exploit v3 was written to close, walking back in through dialogue.

1. What the pipeline generates

Six retry reads, one per phase gate and register combination. A row carries the gate the player died at, the phase, the charge band, the line, its word count, how many attempts it took, which rules fired along the way, and where the line came from.

The output is a UE5 DataTable, `out/DT_RetryReads.json` and `.csv` with a `USTRUCT` header, which is the same integration path Assignments 3 and 4 used. The engine loads it on day one.

2. The rule the Evaluator enforces

Five rules, each traceable to a line in the GDD. R1 is the one this assignment is about.

RULE	WHAT IT REJECTS	WHERE IT COMES FROM
R1_TENSE	future or intent language	§3 exploit 1, past-tense diagnosis only, never intent
R2_PRIOR_REF	a line that does not name the gate she lost at	§3, the first read after a loss references the failed run
R3_BUDGET	more than 12 words	§1, the reads are barks, the exemplar is 8 words
R4_INVENTED_NUMBER	a figure absent from this round's telemetry	§5, every agent output passes an engine-side validity check
R5_JOURNEY_LEAK	journey vocabulary the robot cannot know	§4, the Nemesis never receives the journey

All five are deterministic Python. No second model call decides whether a line passed, which is what makes the catches reproducible and testable in both directions. `tests/test_ger.py` gives every rule a line it must reject and a line it must pass, and it checks that the authored fallbacks obey the same rules the model is held to.

The Generator never sees the rule list. It is given the voice and the telemetry and nothing else. A generator that can read the rules writes to the rules, and then the Evaluator's catches are theatre. Every failure recorded in `evidence/ger_trace.md` is a break the model made on its own. The Refiner is the only stage that sees rules, and it sees only the ones that fired.

3. What it caught

Run of 18 August against `claude-haiku-4-5`. Six of six first drafts broke a rule. Five were repaired inside the loop. One escalated to the breaker.

R1 never fired. The rule this pipeline was built around, the read-line oracle, caught nothing. Haiku did not once write "I'll seal the fence this time". The failure mode I designed against did not occur, and a report that claimed otherwise would be describing a run that did not happen.

R2 fired on all six. Every first draft described the movement statistics and omitted the gate she died at. The generator's opening line for the strained yard case was `Quartering pattern detected. Limping gait. Collar signature. Predictable. Closing.` That is a good Nemesis bark and a useless retry read, because it does not reference the run she just lost, which is the one thing GDD §3 promises this line will do. The refiner returned `Yard failure. Balanced approach vectors. Strained gait deterioration. Predictable. Closing.`

So the answer to whether it caught something I would have missed is yes, and it was not what I expected. Reading those six drafts by hand I would have nodded at all of them. They sound like the robot. They quietly break the retry promise.

The escalation caught my rule being wrong, not the model. On the confident fence gap case the model wrote `Leftward bias forty-five percent`. The left frequency in that case is 0.45, so forty-five percent is a true reading of the telemetry. R4 rejected it anyway, because my allowed-number set holds counts out of twenty and not percentages, and `forty-five` parses as forty. The refiner had nothing real to fix. It restated the same true statistic twice, R4 rejected it twice more, and the breaker stopped at three attempts and shipped the authored fallback.

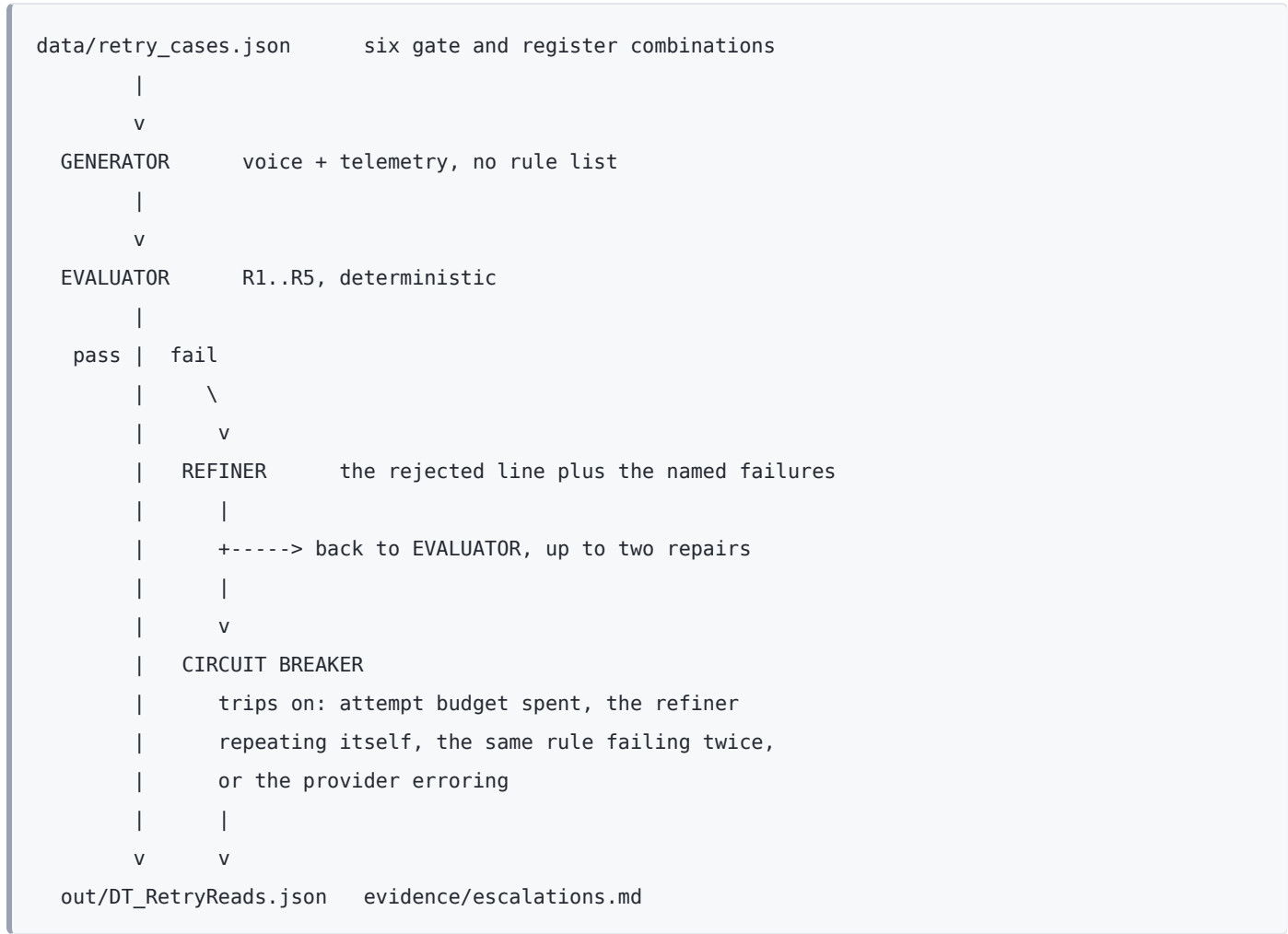
That is the breaker earning its place. A loop without one keeps paying for calls that cannot succeed, because the objection it is answering is not answerable. The row still shipped, tagged `authored_fallback`, so the DataTable is complete and nothing downstream has to guess where the line came from.

Both R4 rejections were the same flaw, and the flaw is mine. Forty-five percent is the left frequency of 0.45 written as a percentage. On the first live run the model wrote `Right-up bias seventy percent`, which I read as invented until I checked: right and up are 0.35 each in that case, so seventy is their sum. R4 holds a flat list of the numbers present in the telemetry, and the model does arithmetic on the telemetry, so a correctly derived figure is indistinguishable from a fabricated one. R4 did not catch a single false statistic in either run.

What the first run did expose was that the list was also incomplete. "Seventy" was missing from my spelled-number vocabulary while sixty and eighty were present, so that line passed for the wrong reason. Twenty-four passing tests had not found the hole. One live call did. The vocabulary is fixed with a regression test and the suite is now 26, and the deeper problem, that R4 cannot do arithmetic, is named in §7 and not solved here.

Full attempt-by-attempt evidence is in `evidence/ger_trace.md` and `evidence/escalations.md` , both emitted by the run rather than written afterwards.

4. How the loop works



The breaker never drops a row. When it trips, the row ships with an authored fallback line and Provenance: `authored_fallback` , so nothing downstream has to guess where a line came from. That is GDD §6 constraint 5 in code: a dropped request costs flavor, never a frame.

Three things make the breaker more than a retry counter. It stops when the Refiner returns the same text twice, because a second identical reply is not going to become a third different one. It stops when the same rule fails on consecutive repairs, which means the model has not understood the objection. And it refuses to retry a provider error at all, because retrying a rejected key burns time without changing anything.

5. Running it

```
python pipeline.py          recorded turns, no key, no network
python pipeline.py --live   real calls against claude-haiku-4-5
python run_live.py          the live run plus the tests, one command
python tests/test_get.py    26 assertions
```

Python 3.8 or newer.

To run it live, supply your own key in the same terminal:

```
PowerShell: $env:ANTHROPIC_API_KEY = "your-key"
cmd.exe:    set ANTHROPIC_API_KEY=your-key
bash or zsh: export ANTHROPIC_API_KEY=your-key
```

No key is stored anywhere in this archive and none should ever be.

7. Known limits

Six cases is a thin corpus. Three gates and three registers give nine combinations and this run covers six of them, chosen to spread across all three gates and all three registers rather than to be exhaustive.

R4 cannot do arithmetic. It holds the numbers present in the telemetry and nothing derived from them, so the left frequency of 0.45 is held as the count nine and the model's "forty-five percent" is rejected, and right plus up is rejected as well. That strictness caused the only escalation in the run and both R4 rejections were of true figures. The fix is to expand the allowed set with the percentage form of every frequency and the pairwise sums, or to hand R4 the raw telemetry and let it verify a claim instead of matching a list. Neither is in this submission. I found it while reviewing thirty minutes before the deadline.

R1 is a token list. It catches the future and intent constructions a model actually reaches for, and a determined paraphrase could still smuggle intent past it. On this run it caught nothing at all, which is worth saying plainly: the rule is tested and correct and the model simply does not make that mistake. It stays because the GDD names the exploit and a different model, or a longer corpus, may well reintroduce it.

The register field is carried through the table and used to shape the prompt, and no rule enforces it. Nothing yet rejects a chatty line written under a strained charge. That is the next rule to write.

The word budget of 12 is mine rather than the GDD's. The document gives an exemplar of 8 words and calls the reads barks, and 12 is my reading of the headroom around that.

8. Where each stage lives

ger/generate.py	GENERATOR	voice and telemetry, no rule list
ger/rules.py	EVALUATOR	R1..R5, deterministic, no model call
ger/refine.py	REFINER	the rejected line plus the rules that fired
ger/breaker.py	CIRCUIT BREAKER	stop conditions and the authored fallbacks
pipeline.py		the loop that runs the four of them, and the writers
ger/llm.py		raw urllib provider, live and replay, no framework
data/		the six gate and register cases
out/		DT_RetryReads.json, .csv and the USTRUCT header
evidence/		ger_trace.md and escalations.md, emitted by the run
transcript/		23 recorded Haiku turns, keyed by prompt hash
tests/		26 assertions
DECLARATION.txt		the Pre-Build Declaration on its own